

MongoDB Operators Using Mongoose

Before using operators, create Schema and Model.

Schema

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  course: String,
  city: String
});
```

Model

```
const Student = mongoose.model("Student", studentSchema);
```

Comparison Operators

Q1. How do we find students whose age is greater than 20?

Operator: \$gt (Greater Than)

```
const students = await Student.find({
  age: {
    $gt: 20
  }
});
```

Q2. How do we find students whose age is less than 25?

Operator: \$lt (Less Than)

```
const students = await Student.find({
  age: {
    $lt: 25
  }
});
```

Q3. How do we find students whose age is greater than or equal to 21?

Operator: \$gte

```
const students = await Student.find({
  age: {
    $gte: 21
  }
});
```

Q4. How do we find students whose age is less than or equal to 25?

Operator: \$lte

```
const students = await Student.find({
  age: {
    $lte: 25
  }
});
```

Q5. How do we find students whose age is not equal to 21?

Operator: \$ne

```
const students = await Student.find({
  age: {
```

```
    $ne: 21
  }
});
```

Membership Operators

Q6. How do we find students enrolled in MERN or Java?

Operator: \$in

```
const students = await Student.find({
  course: {
    $in: ["MERN", "Java"]
  }
});
```

Q7. How do we find students who are not enrolled in Python?

Operator: \$nin

```
const students = await Student.find({
  course: {
    $nin: ["Python"]
  }
});
```

Logical Operators

Q8. How do we find students whose age is greater than 20 and course is MERN?

Operator: \$and

```
const students = await Student.find({
  $and: [
    { age: { $gt: 20 } },
```

```
    { course: "MERN" }  
  ]  
});
```

Q9. How do we find students whose course is MERN or Java?

Operator: \$or

```
const students = await Student.find({  
  $or: [  
    { course: "MERN" },  
    { course: "Java" }  
  ]  
});
```

Q10. How do we find students who are neither from Delhi nor Mumbai?

Operator: \$nor

```
const students = await Student.find({  
  $nor: [  
    { city: "Delhi" },  
    { city: "Mumbai" }  
  ]  
});
```

Element Operators

Q11. How do we find documents where email field exists?

Operator: \$exists

First add email field in schema.

```
const students = await Student.find({  
  email: {
```

```
    $exists: true
  }
});
```

Regular Expression Queries

Q12. How do we find students whose name starts with S?

```
const students = await Student.find({
  name: /^S/
});
```

Examples:

- Shahid
- Sameer
- Salman

Q13. How do we find students whose name ends with d?

```
const students = await Student.find({
  name: /d$/
});
```

Examples:

- Shahid
- Farid

Q14. How do we find students whose name contains "ha"?

```
const students = await Student.find({
  name: /ha/
});
```

Examples:

- Shahid
 - Shahrukh
-

Combining Operators

Q15. How do we find MERN students whose age is between 20 and 25?

```
const students = await Student.find({
  course: "MERN",
  age: {
    $gte: 20,
    $lte: 25
  }
});
```

Q16. How do we find Java or MERN students whose age is greater than 18?

```
const students = await Student.find({
  age: {
    $gt: 18
  },
  course: {
    $in: ["Java", "MERN"]
  }
});
```

Most Important Interview Queries

Find students older than 20

```
Student.find({
  age: {
```

```
    $gt: 20
  }
});
```

Find students between age 20 and 30

```
Student.find({
  age: {
    $gte: 20,
    $lte: 30
  }
});
```

Find students from Delhi

```
Student.find({
  city: "Delhi"
});
```

Find students from Delhi or Mumbai

```
Student.find({
  $or: [
    { city: "Delhi" },
    { city: "Mumbai" }
  ]
});
```

Find students enrolled in MERN or Java

```
Student.find({
  course: {
    $in: ["MERN", "Java"]
  }
});
```

```
}  
});
```

Quick Revision

Comparison Operators

```
$gt  
$lt  
$gte  
$lte  
$ne
```

Membership Operators

```
$in  
$nin
```

Logical Operators

```
$and  
$or  
$nor
```

Element Operators

```
$exists
```

Search Operators

```
/regex/
```

These operators are used inside `find()`, `findOne()`, `updateMany()`, `deleteMany()`, and almost every MongoDB query.